SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Technical Presentation

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	**Assessment:**	Assessment Tool 3 – Technical Presentation
Weightage:	35% of CIA	**Total Marks:**	100
CO1 Statement:	*Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.*		
Student Name:	DHIVYADHARSAN M.S	**Reg. No.:**	192511465
Section / Batch:	2025	**Date:**	15/07/2026

Code of Conduct

*I*____ DHIVYADHARSAN M.S_____*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: DHIVYADHARSAN M.S_____

Instructions

- This assessment contains technical presentation. Total: 100 Marks.
- When a student delivers a technical presentation on a data structures topic, evaluation should be based on the following requirements: Concept explanation, Real-world application and Clarity of presentation.
- Demonstrate clear understanding, not just reading slides
- Use of tools: PowerPoint / Google Slides.
- Duration: 7–10 minutes presentation + 3 minutes Q&A.

Section / Topic	Focus	Q Nos	Marks
Data Structure	Real-Time Applications	1	100
	GRAND TOTAL:	**100 Marks**	

Annexure A – Technical Presentation

A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency. (100 Marks)

(OR)

A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation. (100 Marks)

(OR)

A metro rail network maps stations and routes to determine reachable stations from a source station. Suggest a suitable data structure for representation and justify your answer. (100 Mark)

(OR)

A calculator application evaluates arithmetic expressions entered by users. Recommend a suitable data structure and justify based on expression processing efficiency. (100 Marks)

(OR)

A cafeteria token system serves customers strictly in the order of arrival. Identify the suitable ADT and justify why it ensures fairness. (100 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates deep and accurate understanding of data structure with insights and connections	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Content Organization	30	Logical, well-structured, smooth flow; strong introduction and conclusion	Mostly organized with clear flow	Some organization but lacks clarity or transitions	Disorganized, difficult to follow
Technical Depth	20	Includes algorithms, complexity analysis, and advanced insights	Includes algorithm and basic complexity analysis	Limited technical details; missing depth	Minimal or no technical content
PowerPoint Slides	20	Excellent design, clear visuals, enhances understanding	Good slides with minor issues	Basic slides; somewhat cluttered or unclear	Poor slides or no visual support
Communication Skills	10	Clear, confident, engaging delivery; excellent articulation	Clear delivery with minor hesitation	Some hesitation; unclear in parts	Poor communication; difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Expression Evaluation in Calculator Applications

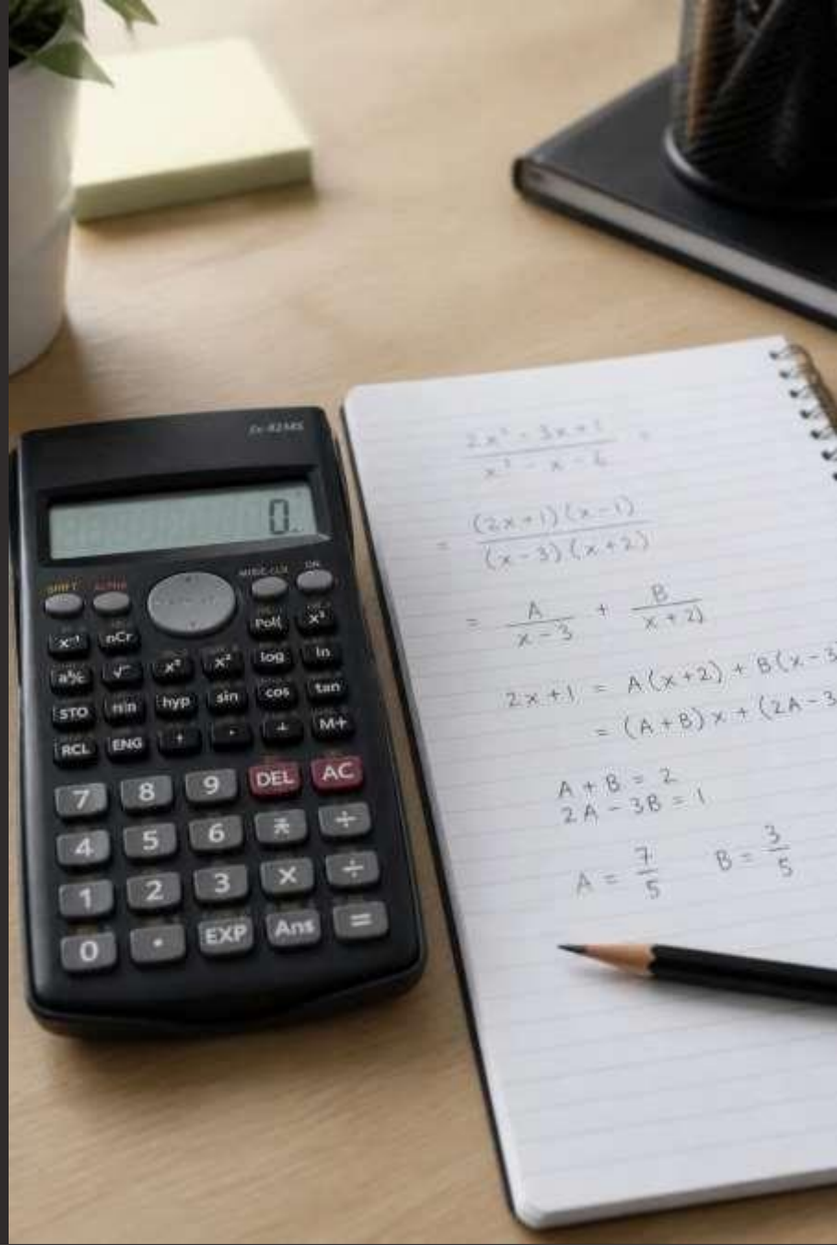
Recommendation of a Suitable Data Structure Based on Expression Processing Efficiency

.

Presented by: M.S DHIVYADHARSAN — Roll No: 192511465 — Dept. of CSE, SIMATS Engineering

Abstract

Calculator apps must parse and evaluate user-entered arithmetic expressions while preserving correctness, precedence and performance. This deck argues that the Stack is the most suitable data structure: it maps directly to nested scopes (parentheses), enables linear-time infix→postfix conversion and one-pass evaluation, and keeps memory usage minimal.



Problem Statement

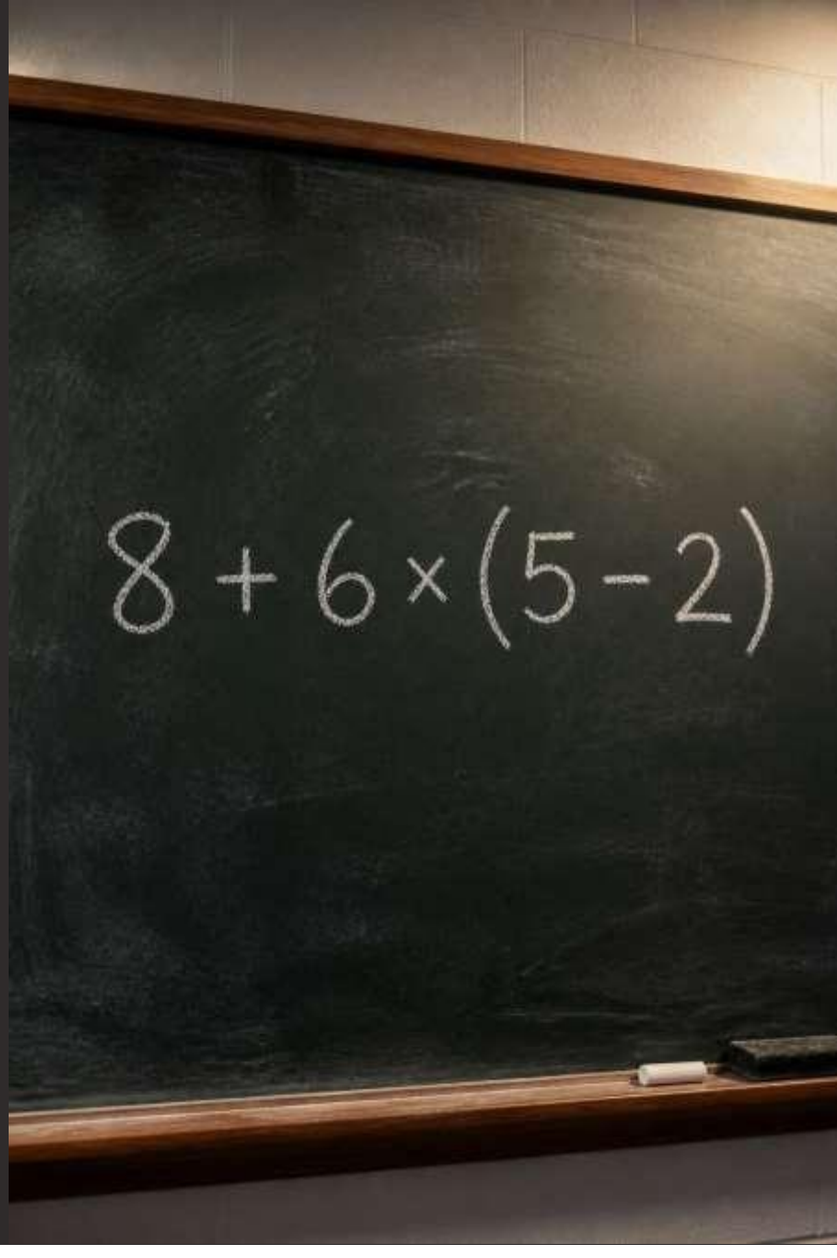
Requirement: evaluate expressions correctly and efficiently. Example: $8 + 6 \times (5 - 2)$

Core challenges

- Correct operator precedence ($\times$ before $+$)
- Proper handling of nested parentheses
- Accurate numeric evaluation (including unary ops, associativity)
- Low latency and bounded memory use

Success criteria

- Correctness across inputs
- O(n) time for typical algorithms
- Simple, maintainable implementation



Recommended Data Structure —Stack

The Stack (LIFO) is the ideal primitive for expression evaluation: it mirrors nested scopes and lets you defer operator application until operand and precedence context are known.

Direct mapping

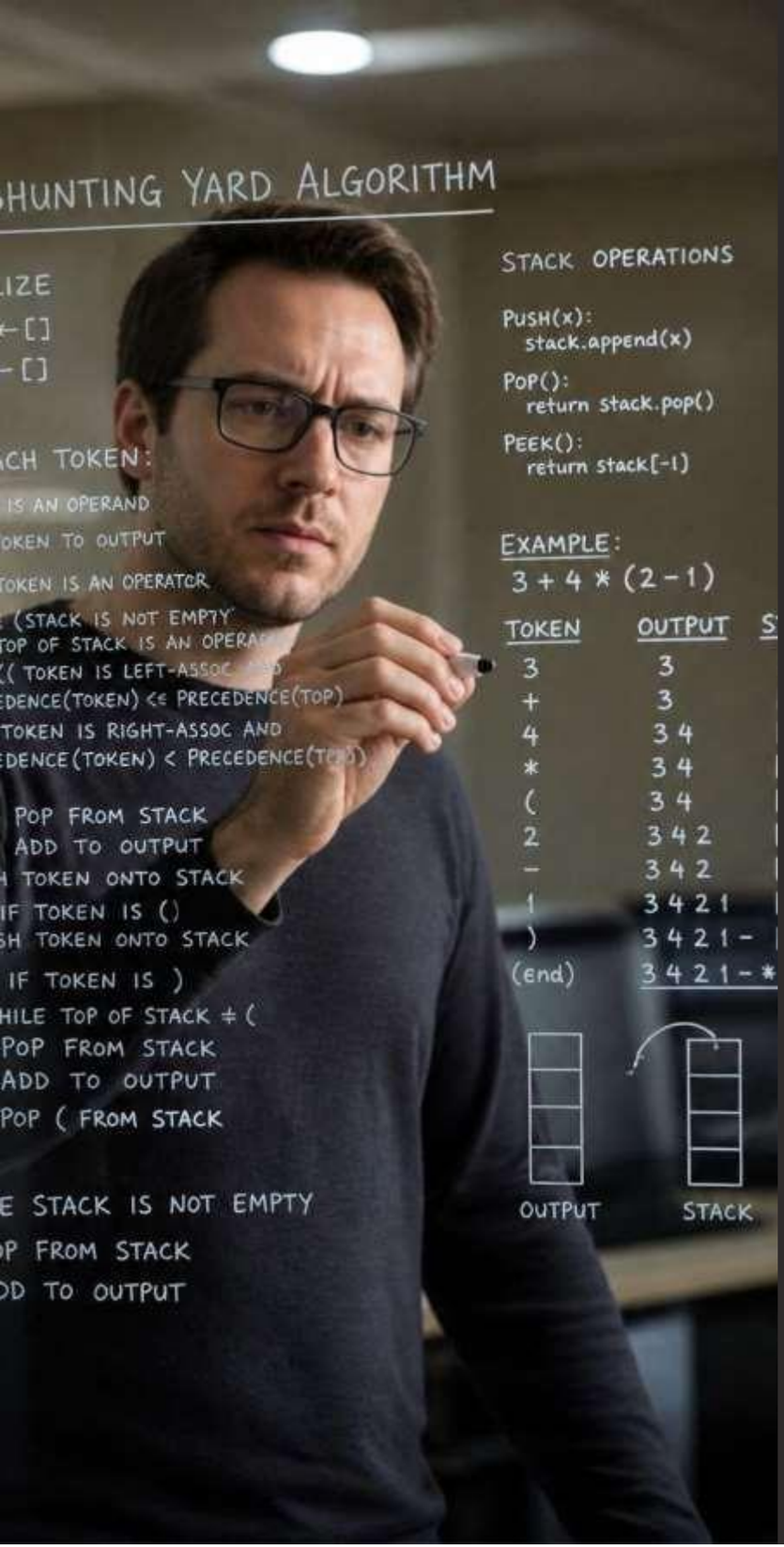
Parentheses and nested sub-expressions push and pop frames naturally.

Performance

Infix→postfix conversion (Shunting Yard) and postfix evaluation both run in O(n).

Simplicity

Small, well-understood API: push, pop, peek →easy to test and reason about.



Working Process (Practical)

Typical pipeline (linear scan):

Tokenize

Split input into numbers, operators, parentheses, and unary markers.

Infix →Postfix (Shunting Yard)

Use operator stack +output queue; apply precedence and associativity rules when encountering operators or parentheses.

Evaluate Postfix

Scan tokens: push operands; on operator, pop operands, compute, push result.

Edge cases

Unary operators, implicit multiplication, function calls, and numeric precision—handled with token rules or small parser extensions.

Time complexity: push/pop O(1); full evaluation O(n). Space: O(n) worst-case for stacks/queues proportional to token count.

Advantages & Applications

Key advantages: fast evaluation, low implementation complexity, robust handling of nested constructs, and easy extension for functions and unary operators.

Compiler Design

Stacks underpin expression parsing in compilers (AST construction, operator reduction).

Scientific Calculators

Nested functions and precedence resolved efficiently using stacks.

Runtime Interpreters

Interpreters evaluate expressions at runtime with stack-based VM semantics (push/pop operands).

Conclusion & Next Steps

Stack-based processing (tokenize → shunting-yard → postfix evaluation) provides a compact, O(n)-time solution that satisfies correctness, performance and memory constraints for calculator applications. Next steps for implementation: robust tokenizer, handle unary/implicit ops, add tests for edge cases, and consider using a stack-backed VM for complex expressions.

Code hint: implement operator table with precedence & associativity; use two stacks (operators, operands) or convert to postfix then evaluate.

Thank you